

CLAUSIO

Database Design Specification

The bridge between the business domain and the Prisma schema

Version 1.0

Derived from the Clausio System Design Blueprint and the Clausio Domain & Business Rules
July 10, 2026

Purpose & Scope

This Database Design Specification (DDS) defines the data model that will implement the Clausio business domain. It is the bridge between two approved, authoritative documents — the System Design Blueprint and the Domain & Business Rules — and the Prisma schema that developers will write next.

Its goal is verification, not code: to confirm, before a single line of schema is written, that the database design fully supports every business rule (BR-1 through BR-21) and every lifecycle defined in the domain. Where the design refines or extends the original Blueprint tables to satisfy the later-approved business rules, those decisions are called out explicitly and collected in Appendix A.

This document intentionally contains no `schema.prisma`, no SQL, and no migrations. It specifies entities, enums, relationships, constraints, indexes, and strategies in design language so they can be reviewed and signed off. The final section maps each business concept to the Prisma model name that will represent it.

A note on Workspace: as established in the Domain & Business Rules (improvement to the Workspace concept), a Workspace is a conceptual environment, not a stored entity. There is no workspace table and no Prisma model for it. The Organization is the actual owner of all data, and “the Organization's Workspace” simply means the set of rows scoped to that Organization.

Contents

Purpose & Scope	2
Contents	3
1. Database Design Principles	4
2. Complete Entity List	7
3. Enums.....	11
4. Relationships	16
5. Constraints	18
6. Indexes	20
7. Soft Delete Strategy	21
8. Audit Strategy	22
9. Seed Strategy	23
10. Prisma Mapping	24
Appendix A. Reconciliation with the Blueprint	25

(In Microsoft Word, right-click this table and choose "Update Field" to populate page numbers.)

1. Database Design Principles

These principles govern every entity in the model. They exist so that the business rules are enforced structurally by the database wherever possible, rather than relying solely on application code.

1.1 Multi-tenancy strategy

Clausio uses a single shared database with a shared schema, where tenant data is separated by row. Every tenant-owned record carries an `organization_id` column identifying the Organization that owns it. One physical database serves all Organizations; there is no database-per-tenant or schema-per-tenant split.

This model is the right fit for Clausio's scale and team size: it is simple to operate, migrate, and back up, while still supporting the strict isolation demanded by BR-5 and BR-18. It relies on disciplined tenant scoping (below) to keep tenants apart.

1.2 Tenant isolation

Isolation is enforced in depth. First, every authenticated request is bound to exactly one Organization (BR-20), and every tenant-scoped query must filter by that `organization_id` — this filter is applied centrally, not left to individual call sites. Second, database row-level security keyed on a per-session organization identifier acts as a backstop, so that even a query that forgot its filter cannot read another tenant's rows.

Tenant isolation must never rely on frontend filtering. The client is not a trust boundary: hiding another Organization's data in the UI is not isolation, since any direct request to the backend could otherwise bypass it. Every backend service, repository, and database query must explicitly enforce organization scoping before executing any operation — organization boundaries are enforced server-side, for every authenticated request, regardless of what the calling client sends or omits.

- **Platform is separate.** Platform users are a separate identity domain and are deliberately not tenant-scoped; their separation from customer work is governed by BR-21 (see the PlatformUser entity).
- **Witnesses are scoped, not members.** Witnesses are not tenant members (BR-4). Their access is mediated entirely by a single WitnessInvitation row scoped to one Contract, never by organization membership (BR-8).

1.3 UUID strategy

Every primary key is a UUID rather than a sequential integer. UUIDs are non-guessable, which matters because identifiers appear in URLs (contract links, witness links) and must not allow one Organization to enumerate another's records. They also carry no cross-tenant ordering information and are safe to generate on either side of the wire.

A time-ordered UUID variant (UUIDv7) is recommended over fully random UUIDv4 so that primary-key indexes retain insertion locality and stay efficient as tables grow; the tradeoff (a small amount of embedded timestamp information) is acceptable for these identifiers.

1.4 Soft delete strategy

Clausio almost never destroys data. “Deletion” takes one of three forms: a soft delete (a nullable `deleted_at` timestamp; the row is hidden from normal queries but retained), a terminal status transition (for example an Organization becomes ARCHIVED or a User becomes DEACTIVATED), or — for append-only records — no deletion at all. Physical removal is reserved for exceptional, audited operations. Contracts specifically must be soft-deleted only (BR-7). Section 7 states the strategy for every entity.

1.5 Audit strategy

Every data-changing action writes an event to a single append-only `audit_logs` table (BR-14), and those events are immutable — never updated or deleted, enforced by a database rule that rejects modification (BR-13). The audit trail outlives the records it describes: soft-deleting a contract or deactivating a user never removes their history. Section 8 details captured fields and redaction rules.

1.6 Naming conventions

The database and the Prisma layer use consistent, predictable names, mapped to each other explicitly.

- **Tables:** plural snake_case (organizations, contract_notes, ai_analyses).
- **Columns:** snake_case; primary key is always id; foreign keys are <referenced_singular>_id (organization_id, contract_id, uploaded_by_user_id).
- **Suffixes/prefixes:** timestamps end in _at (created_at, updated_at, deleted_at, expires_at, used_at); booleans start with is_ or has_ (is_revoked).
- **Enums:** PascalCase type name with UPPER_SNAKE values (OrganizationStatus.OWNER_ASSIGNED).
- **Prisma models:** PascalCase singular (Organization, ContractNote, AIAnalysis), each mapped to its plural snake_case table. Prisma fields are camelCase, mapped to snake_case columns.

1.7 Timestamp strategy

All timestamps are stored in UTC using a timezone-aware type. Mutable entities carry `created_at` and `updated_at`; append-only entities (audit events, completed AI analyses) carry only `created_at`, which reinforces their immutability by giving them no “last modified” concept. Soft-deletable entities add `deleted_at`. Domain-specific moments have their own columns (`expires_at`, `used_at`, `last_login_at`, `owner_assigned_at`). Display in a user's local time is derived from the Organization's configured timezone setting; storage is always UTC.

1.8 Concurrency strategy

Mutable entities — Contract above all, since it is edited by multiple members and updated by AI processing over time — should support optimistic concurrency control using a version number or an equivalent mechanism (such as comparing `updated_at`). Each update checks that the version it is writing against still matches the current row, and increments it on success.

This prevents two users from silently overwriting each other's changes: if two people edit the same Contract at nearly the same time, the second write is rejected rather than quietly discarding the first person's changes, and the application can prompt them to reload and reapply their edit. This is an

implementation guideline for the Prisma layer and application code, not a schema redesign — it is satisfied by a single additional version-tracking field on the entities that need it.

2. Complete Entity List

Each entity below is described by why it exists, which business rules require it, who owns it, and its lifecycle. The model comprises ten stored entities plus two optional/normalization entities (Tag and ContractTag). Two domain concepts intentionally have no table of their own:

- **Workspace** — a conceptual view of an Organization's owned data, not a stored entity. No table, no model (see Purpose & Scope, and Section 10).
- **Witness** — not a persistent identity. A Witness has no account (BR-4); their access exists only as a WitnessInvitation row scoped to one Contract.

2.1 Organization

Why it exists — The root of every tenant. It represents the customer entity and is the owner of all data beneath it; it anchors tenant isolation and the single-Owner rule.

Business rules — BR-1, BR-3, BR-5, BR-6, BR-18, BR-20.

Owned by — The Platform. Organizations are created by a Platform Super Admin, who also assigns the first Owner.

Lifecycle — OrganizationStatus: CREATED → OWNER_ASSIGNED → ACTIVE → SUSPENDED → ARCHIVED (with SUSPENDED → ACTIVE reactivation). Never hard-deleted; ARCHIVED is terminal.

2.2 OrganizationSettings

Why it exists — Holds the Organization's configurable business settings (logo, timezone, language, AI provider, notification preferences, contract retention policy, branding) as typed fields rather than an opaque blob, so each setting can be validated and queried. Organization name lives on the Organization itself.

Business rules — Supports the Organization Settings domain section; retention policy interacts with BR-7 and BR-13; AI provider selection is still bound by all AI rules (BR-10–BR-12).

Owned by — Exactly one Organization (one-to-one).

Lifecycle — Created together with the Organization; updated over time; never independently deleted.

2.3 PlatformUser

Why it exists — Represents Clausio's own operators — the Platform Super Admin and Support Engineer roles — as a separate identity domain from tenant Users. This is what makes the platform/customer separation real in the data model.

Business rules — BR-21 (platform users operate Clausio, not customers' day-to-day work); underpins the Platform Roles section.

Owned by — The Platform itself. A PlatformUser has no organization_id and is never a member of any Organization.

Lifecycle — PlatformUserStatus: ACTIVE → SUSPENDED → DEACTIVATED. Support access elevation is time-bound and audited rather than being a persistent state.

2.4 User

Why it exists — A person with an identity and a role inside one Organization. Users perform all day-to-day contract work.

Business rules — BR-1 (exactly one Owner), BR-3 (belongs to one Organization), BR-16 (only Admin/Owner delete), BR-17 (unique email), BR-19 (Owner cannot be suspended/deactivated while Owner).

Owned by — Exactly one Organization.

Lifecycle — UserStatus: INVITED → ACTIVE → SUSPENDED → DEACTIVATED (with SUSPENDED → ACTIVE reactivation). Role is UserRole: OWNER, ADMIN, or INTERNAL. Witnesses are deliberately excluded from this entity (BR-4).

2.5 Invitation

Why it exists — Represents a pending offer of membership. It drives the INVITED state and the invitation flow (token generated → delivered → accepted), and it is the mechanism that lets Clausio have no public registration — the only way to join is by Invitation.

Business rules — BR-3, BR-17; realizes the User INVITED lifecycle state and the invitation-flow requirement.

Owned by — The Organization; created by an Owner or Admin (the inviter).

Lifecycle — InvitationStatus: PENDING → ACCEPTED / EXPIRED / REVOKED. Time-limited via expires_at; an accepted Invitation results in an ACTIVE User.

- **Single-use.** an Invitation's token is valid for exactly one acceptance; once used, it cannot be used again.
- **Accepted invitations become history.** an accepted Invitation is never deleted — it becomes a historical record of how and when that member joined, permanently linked to the resulting User.
- **Expired invitations remain history.** an Invitation whose expires_at has passed is marked EXPIRED and kept, not removed, preserving a record that access was offered and not taken up.
- **Revoked invitations remain history.** an Invitation withdrawn by an Owner or Admin is marked REVOKED and kept, preserving a record of who withdrew it and when.
- **Re-inviting a former member.** if a person who has left an Organization (DEACTIVATED) is invited again, this always creates a brand new Invitation with a new token; the old, historical Invitation is never reopened or reused.
- **Old tokens are permanently spent.** a token belonging to an ACCEPTED, EXPIRED, or REVOKED Invitation can never be reused to join — only a fresh PENDING Invitation grants access.

Preserving every Invitation as a permanent historical record, rather than deleting it once resolved, matters for both security and auditing: it lets the Organization answer, at any later point, exactly who was invited, by whom, when, under what role, and how the invitation was resolved — accepted, left to expire, or revoked — which is essential for reconstructing how someone gained (or was denied) access and for spotting any unusual invitation activity after the fact.

2.6 Contract

Why it exists — The central artifact of the product: an uploaded agreement with extracted text, metadata, notes, and analyses.

Business rules — BR-6 (one Organization for life), BR-7 (soft delete only), BR-15 (remains usable even if extraction fails), BR-16, BR-18.

Owned by — Exactly one Organization; `uploaded_by` references the User who created it.

Lifecycle — Contract deliberately tracks two independent concepts rather than one combined status.

- **Business Status** — `ContractBusinessStatus`: DRAFT → UNDER_REVIEW → COMPLETED → ARCHIVED. This is the workflow a human cares about — has the contract been looked at, is it settled, is it retired — and it is what drives permissions, dashboards, and reporting.
- **Processing Status** — `ContractProcessingStatus`: PENDING_EXTRACTION → EXTRACTION_COMPLETED → AI_PENDING → AI_COMPLETED, with a FAILED value reachable from either processing step. This tracks internal technical operations — whether text has been pulled from the document and whether AI analysis has run — and is largely invisible to end users.

The two fields evolve independently: a Contract can sit in DRAFT while its Processing Status moves through PENDING_EXTRACTION, EXTRACTION_COMPLETED, AI_PENDING, and AI_COMPLETED in the background, and a failed extraction or AI run (Processing Status FAILED) never blocks or reverts the Business Status, consistent with BR-15. Witness sharing is likewise independent of both: it is modeled entirely by the `WitnessInvitation` entity's own lifecycle (Section 2.8), so a Contract does not need its own “Witness” business state — a Contract can have an active witness invitation while UNDER_REVIEW, COMPLETED, or even still in DRAFT. Soft-deletable at any point via `deleted_at`, independent of either status.

2.7 ContractNote

Why it exists — Human annotations attached to a Contract during review.

Business rules — BR-18 (visible only within the owning Organization).

Owned by — A Contract (and therefore its Organization); `author_id` references the authoring User.

Lifecycle — Created during review; editable by its author or an Admin (edits are audited); soft-deleted with, and hidden alongside, its Contract.

2.8 WitnessInvitation

Why it exists — Grants an external party scoped, expiring, single-use, read-only access to exactly one Contract without an account. This is the concrete form the Witness concept takes in the data model. (In the Blueprint this table is named `witness_tokens`.)

Business rules — BR-4 (no account, no membership), BR-8 (single contract, read-only, until expiry/revocation), BR-9 (single-use, time-limited).

Owned by — A Contract (and its Organization); `issued_by` references the Admin/Owner who created it.

Lifecycle — `WitnessStatus`: ISSUED → USED / EXPIRED / REVOKED. Stores a `token_hash` (never the raw token), `expires_at`, `used_at`, and `is_revoked`.

2.9 AIAnalysis

Why it exists — A read-only, generated interpretation of a Contract's text — a summary, risk scan, or clause question — stored permanently and attached to the Contract.

Business rules — BR-10 (never edits the Contract), BR-11 (carries a disclaimer, no legal advice), BR-12 (answers only from the Contract text).

Owned by — A Contract (and its Organization); created_by references the requesting User.

Lifecycle — AIAnalysisStatus: PENDING → COMPLETED / FAILED. Immutable once COMPLETED; records its analysis type and the pinned model_version.

2.10 AuditLog

Why it exists — The immutable, append-only history of every meaningful action in an Organization.

Business rules — BR-13 (immutable), BR-14 (every data-changing action logged); supports BR-2/BR-19 (ownership changes) and witness-access logging (BR-8).

Owned by — The Organization (org-scoped). The actor may be a User, a PlatformUser (for authorized platform actions), or the system.

Lifecycle — Insert-only. Never updated or deleted; retained beyond the soft-deletion or archival of the records it references.

2.11 Tag and ContractTag (optional / normalized)

The Blueprint stores contract tags as a simple string array on the Contract, which is the recommended default. If a shared, governed tag vocabulary is later required, tags can be normalized into a Tag entity (org-scoped) joined to Contract through a ContractTag table — the model's one genuine many-to-many. Both options are described here so the choice can be made explicitly; the array form needs neither extra entity.

3. Enums

Every status and category below is a database enum. Storing these as enums (rather than free text) makes the lifecycles from the Domain & Business Rules enforceable at the data layer.

3.1 OrganizationStatus

Value	Meaning
CREATED	Provisioned but not yet operational; no Owner assigned; empty Workspace reserved.
OWNER_ASSIGNED	A single Owner has been designated; the Organization can be configured but is not yet in full use.
ACTIVE	Full normal operation; members, contracts, AI, and witness access are all available per role.
SUSPENDED	Temporarily frozen (by the Platform or at the Owner's request); data preserved, actions blocked; reversible to ACTIVE.
ARCHIVED	Permanently retired; Workspace closed to new activity; all records retained. Terminal.

3.2 UserStatus

Value	Meaning
INVITED	A membership Invitation exists but has not been accepted; the person cannot yet act.
ACTIVE	A full member acting within their role.
SUSPENDED	Temporarily disabled; account and history preserved; reversible to ACTIVE (reactivation).
DEACTIVATED	Retired from the Organization; access ends permanently; authored history retained. Terminal.

3.3 UserRole

Value	Meaning
OWNER	The single principal accountable for the Organization; exactly one per Organization (BR-1); transferable only (BR-2).
ADMIN	Full management of users, contracts, settings, witness links, and audit review; may soft-delete contracts (BR-16).
INTERNAL	Day-to-day contract work; cannot manage users or delete contracts.

Note: the Blueprint's original role enum included a witness value. It is deliberately omitted here because a Witness is never a User (BR-4); witness access is modeled solely through WitnessInvitation. See Appendix A.

3.4 PlatformUserRole & PlatformUserStatus

Value	Meaning
SUPER_ADMIN	Highest authority over Clausio itself: creates Organizations, assigns first Owners, suspends/archives Organizations, recovers ownership, manages platform users and analytics.
SUPPORT_ENGINEER	Limited platform support: troubleshoots Organizations, views logs, diagnoses issues; elevated access only when explicitly authorized.

Value	Meaning
ACTIVE	Platform operator in good standing.
SUSPENDED	Temporarily disabled platform account.
DEACTIVATED	Retired platform account; terminal.

3.5 InvitationStatus

Value	Meaning
PENDING	Issued and awaiting acceptance within its validity window.
ACCEPTED	Used in time; a corresponding ACTIVE User now exists.
EXPIRED	Validity window passed without acceptance; no longer usable.
REVOKED	Withdrawn by an Owner or Admin before acceptance.

3.6 ContractBusinessStatus & ContractProcessingStatus

Contract intentionally has two independent status enums rather than one. ContractBusinessStatus represents the contract's business lifecycle — the progress a human tracks. ContractProcessingStatus represents internal technical operations — extraction and AI work happening behind the scenes. The two fields are updated by different parts of the system and must be able to change on their own schedule; a slow or failed background process should never appear to “undo” business progress a reviewer has already made.

Value	Meaning
DRAFT	Uploaded and not yet finalized by a reviewer; the default state for a new Contract regardless of how far extraction or AI processing has progressed.
UNDER_REVIEW	A human is actively working the contract — reading analyses, adding notes, confirming metadata, and optionally sharing it with a Witness for confirmation.
COMPLETED	Review is finished; obligations are understood and recorded; a live, settled business record.
ARCHIVED	Retired from day-to-day view but retained in full (soft-archived; never destroyed, per BR-7).

Value	Meaning
PENDING_EXTRACTION	Document stored; readable text has not yet been pulled from it.
EXTRACTION_COMPLETED	Readable text obtained (or supplied manually per BR-15); ready for AI.
AI_PENDING	One or more AI Analyses have been requested and are running against the extracted text; the Contract itself is unchanged (BR-10).
AI_COMPLETED	The most recently requested AI Analysis has finished successfully.
FAILED	Either extraction or an AI run did not complete successfully; retried or handled without affecting Business Status.

Why two fields: keeping these independent means a Contract can be DRAFT while its Processing Status quietly advances through extraction and AI analysis in the background, and a FAILED processing attempt never forces the business workflow backward or blocks a reviewer from proceeding. Business Status changes only on a person's action (or an explicit rule such as archival); Processing Status changes only as a result of technical operations completing, failing, or being retried. See also Section 2.6.

Related enum — ContractLegalState (recommended, optional): the Blueprint also tracks a contract's legal validity (Draft, Active, Expired, Terminated). This remains a third, distinct axis from both statuses above — it describes the agreement's real-world validity, not Clausio's internal workflow or processing — and is best captured as a separate optional field. Recorded here for the design review; see Appendix A.

3.7 WitnessStatus

Value	Meaning
ISSUED	Created and delivered; not yet accessed; valid until expiry.
USED	Accessed once (used_at recorded); single-use per BR-9.
EXPIRED	Past its expires_at; access denied.
REVOKED	Manually revoked by an Admin/Owner before expiry (is_revoked).

3.8 AIAnalysisType & AIAnalysisStatus

Value	Meaning
SUMMARY	Plain-language summary of obligations, payment, and termination.
RISK	Risk scan flagging categories (auto-renewal, uncapped liability, one-sided indemnification, unusual IP assignment, non-compete, clawback) with severity.
CLAUSE_QUERY	A single question answered strictly from the Contract text (the “chat” / clause investigator type).

Value	Meaning
PENDING	Requested; awaiting the AI response.
COMPLETED	Response stored; immutable thereafter.

Value	Meaning
FAILED	The analysis could not be produced (for example, empty text); recorded so the attempt is visible.

Supporting enum — RiskSeverity (LOW, MEDIUM, HIGH): used if individual risk findings are later stored as structured rows; today severity is embedded in the analysis response text.

3.9 AuditAction

The set of loggable actions, grouped by domain area. Every value corresponds to a data-changing action that must produce an audit event (BR-14).

Area	Action	Meaning
Organization	ORGANIZATION_CREATED	A new Organization was provisioned.
	OWNER_ASSIGNED	The first Owner was designated (CREATED → OWNER_ASSIGNED).
	ORGANIZATION_ACTIVATED	Opened for normal use, or reactivated from SUSPENDED.
	ORGANIZATION_SUSPENDED	Temporarily frozen.
	ORGANIZATION_ARCHIVED	Permanently retired.
	ORGANIZATION_SETTINGS_UPDATED	A setting was changed.
	OWNERSHIP_TRANSFERRED	Ownership moved from one member to another (BR-2).
	OWNERSHIP_RECOVERED	Ownership reassigned by the Platform in an authorized recovery.
User	USER_INVITED	A membership Invitation was issued.
	INVITATION_ACCEPTED	An Invitation was accepted; a User became ACTIVE.
	INVITATION_REVOKED	A pending Invitation was withdrawn.
	USER_SUSPENDED	A member was suspended.
	USER_REACTIVATED	A suspended member was restored.
	USER_DEACTIVATED	A member was retired from the Organization.
	USER_ROLE_CHANGED	A member's role was changed.
Contract	CONTRACT_UPLOADED	A Contract was created.
	CONTRACT_TEXT_EXTRACTED	Text was extracted or manually supplied.
	CONTRACT_METADATA_UPDATED	Title, counterparty, tags, or dates changed.

Area	Action	Meaning
	CONTRACT_BUSINESS_STATUS_CHANGED	A business workflow transition occurred (e.g., DRAFT → UNDER_REVIEW).
	CONTRACT_PROCESSING_STATUS_CHANGED	A technical processing transition occurred (e.g., AI_PENDING → AI_COMPLETED).
	CONTRACT_SOFT_DELETED	A Contract was soft-deleted.
Note	NOTE_ADDED / NOTE_EDITED / NOTE_DELETED	A note was added, edited, or removed.
Witness	WITNESS_INVITATION_CREATED	A scoped witness link was issued.
	WITNESS_ACCESSED	A Witness viewed the Contract (references the invitation).
	WITNESS_INVITATION_REVOKED	A witness link was revoked.
AI	AI_ANALYSIS_REQUESTED / COMPLETED / FAILED	An analysis was requested and then completed or failed.
Platform	PLATFORM_SUPPORT_ACCESS_GRANTED / REVOKED	Temporary elevated support access was granted or ended.

4. Relationships

Every relationship in the model, with its cardinality and the reason it exists. Cardinalities encode business rules directly — for instance, “each User belongs to exactly one Organization” is BR-3 expressed as a relationship.

4.1 One-to-One

Relationship	Why it exists
Organization — OrganizationSettings	Each Organization has exactly one settings record; keeps typed configuration separate from the core tenant row.
Organization — Owner (User)	An Organization references exactly one Owner (owner_user_id). This, with a partial unique rule on the Owner role, enforces “exactly one Owner” (BR-1).

4.2 One-to-Many

Relationship	Why it exists
Organization → User	A tenant has many members; each member belongs to exactly one Organization (BR-3).
Organization → Contract	A tenant owns many contracts; each Contract has one owning Organization for life (BR-6).
Organization → Invitation	A tenant issues many membership invitations.
Organization → AuditLog	Every audit event is scoped to one Organization (BR-18).
Contract → ContractNote	A Contract accumulates many notes; each note belongs to one Contract.
Contract → AIAnalysis	A Contract can have many analyses over time; each analysis belongs to one Contract (BR-10).
Contract → WitnessInvitation	A Contract may be witnessed by several external parties over time; each invitation is scoped to exactly one Contract (BR-8).
User → Contract (uploaded_by)	A member may upload many contracts.
User → ContractNote (author)	A member may author many notes.
User → AIAnalysis (created_by)	A member may request many analyses.
User → WitnessInvitation (issued_by)	An Admin/Owner may issue many witness links.
User → Invitation (invited_by)	An Owner/Admin may send many invitations.
User / PlatformUser → AuditLog (actor)	An actor may be responsible for many audit events; the actor is a User, a PlatformUser, or the system.

4.3 Many-to-Many

Relationship	Why it exists
Contract — Tag (via ContractTag)	Optional and only if tags are normalized: a Contract can carry many Tags and a Tag can apply to many Contracts, enabling a shared, governed vocabulary and efficient filtering. The default Blueprint design keeps tags as a string array, in which case no join table exists.

There are no other many-to-many relationships. In particular, Witness-to-Contract is one-to-many, never many-to-many, because a Witness may access only a single Contract (BR-8) and has no persistent identity to reuse across contracts.

5. Constraints

Constraints turn business rules into guarantees the database itself enforces.

5.1 Unique & composite-unique constraints

Constraint	Purpose / rule
users.email is globally unique	BR-17: a login identity is unique across the entire Platform.
platform_users.email is unique	One identity per platform operator.
one settings row per organization (unique organization_id)	Enforces the 1:1 Organization–OrganizationSettings relationship.
witness_tokens.token_hash is unique	Each witness link resolves to exactly one record; supports safe lookup by hash.
at most one OWNER per organization — partial unique on (organization_id) where role = OWNER	BR-1: guarantees an Organization can never have two Owners. Combined with owner assignment, exactly one exists once ACTIVE.
one pending invite per email per org — unique (organization_id, email) where status = PENDING	Prevents duplicate outstanding invitations to the same person.
tags unique per org — unique (organization_id, name) [normalized option]	Keeps the tag vocabulary clean within a tenant.
contract_tags unique (contract_id, tag_id) [normalized option]	Prevents applying the same tag twice to one Contract.

5.2 Foreign keys & referential actions

Foreign key	References	On delete
organization_settings.organization_id	organizations.id	RESTRICT
users.organization_id	organizations.id	RESTRICT
organizations.owner_user_id	users.id	RESTRICT (BR-19)
invitations.organization_id / invited_by	organizations.id / users.id	RESTRICT
contracts.organization_id / uploaded_by	organizations.id / users.id	RESTRICT
contract_notes.contract_id / author_id	contracts.id / users.id	RESTRICT
ai_analyses.contract_id / created_by	contracts.id / users.id	RESTRICT
witness_tokens.contract_id / issued_by	contracts.id / users.id	RESTRICT
audit_logs.organization_id	organizations.id	RESTRICT
audit_logs.actor_user_id / contract_id	users.id / contracts.id	SET NULL (nullable)
audit_logs.actor_platform_user_id	platform_users.id	SET NULL (nullable)

Why RESTRICT dominates: Clausio does not hard-delete tenant records in normal operation — organizations are archived, users are deactivated, contracts are soft-deleted. RESTRICT prevents accidental cascading destruction. Audit foreign keys use SET NULL (or are simply retained) so that history survives even if a referenced record is ever purged (BR-13).

5.3 Required vs nullable fields

- **Required (NOT NULL):** all id primary keys; every organization_id on tenant-scoped tables; names and titles; every status and role enum; created_at everywhere; witness token_hash and expires_at; audit action, organization_id, and created_at; AI type, prompt, response (when COMPLETED), and model_version.
- **Nullable (by design):** contracts.extracted_text (BR-15 — a Contract stays valid even if extraction fails); contracts.expiration_date; organizations.owner_user_id (null only in the CREATED state, before an Owner is assigned); users.last_login_at; witness used_at; deleted_at on soft-deletable tables; audit old_value (absent on creates) and audit actor references (null for system actions); ai_analyses.tokens_used.

5.4 Cascade vs restrict summary

Situation	Rule
Delete a User who is the current Owner	RESTRICT — blocked until ownership is transferred (BR-19).
Delete/soft-delete a Contract	Notes, analyses, and witness invitations are hidden alongside it; nothing is physically cascaded; audit is retained (BR-7, BR-13).
Delete an Organization	RESTRICT in normal operation (organizations are archived, not deleted). Any true purge is a deliberate, audited ops action, applied in dependency order rather than via automatic cascade.
Reference from an audit event	Never cascades destructively; audit rows outlive their targets.

6. Indexes

Recommended indexes, grouped by the need they serve. Because every tenant query filters by `organization_id` (BR-20), most composite indexes lead with it.

Index (columns)	Serves
users (organization_id, role)	Isolation + fast Owner and member lookups within a tenant.
contracts (organization_id, business_status)	Isolation + filtering the contract list by business workflow stage.
contracts (organization_id, processing_status)	Isolation + finding contracts stuck in or failing background processing.
contracts (organization_id, created_at desc)	Default contract list ordering per tenant.
contracts (organization_id, expiration_date)	Expiry alerts and upcoming-renewal filtering.
contracts full-text index on extracted_text; trigram on title, counterparty	Search across contract content and metadata.
contracts (tags) GIN [array option]	Tag filtering when tags are stored as an array.
ai_analyses (contract_id, type)	Retrieving a contract's analyses by type.
contract_notes (contract_id, created_at)	Listing a contract's notes in order.
witness_tokens (token_hash) unique	Constant-time witness link resolution.
witness_tokens (contract_id)	Listing/managing witness access per contract.
invitations (organization_id, email) where status = PENDING	Enforcing and finding outstanding invitations.
audit_logs (organization_id, created_at desc)	Per-tenant audit browsing (most recent first).
audit_logs (contract_id) / (actor_user_id) / (action)	Per-contract, per-user, and per-action audit views.
partial indexes with where deleted_at is null	Keeping active-set queries fast on soft-deletable tables.

7. Soft Delete Strategy

“Deletion” in Clausio is almost always a soft delete or a terminal status, never physical removal. The table below states the strategy for each entity.

Entity	Strategy	Governing rule
Contract	Soft delete (deleted_at); hidden from normal queries, fully retained.	BR-7
ContractNote	Soft delete; hidden with its Contract; retained for history.	BR-18, BR-14
Organization	No delete — terminal ARCHIVED status instead.	Org lifecycle
User	No delete — terminal DEACTIVATED status instead.	User lifecycle, BR-19
Invitation	No delete — terminal EXPIRED / REVOKED status.	Invitation lifecycle
WitnessInvitation	No delete — terminal EXPIRED / REVOKED status.	BR-8, BR-9
AIAnalysis	Never deleted; immutable record once COMPLETED.	BR-10–BR-12
AuditLog	Never deleted; append-only and immutable.	BR-13
OrganizationSettings	Not independently deleted; lives with its Organization.	—
PlatformUser	No delete — terminal DEACTIVATED status.	BR-21

Never deleted under any circumstance: AuditLog and completed AIAnalysis records. These are the system's memory and evidence; even an exceptional ops-level purge of a tenant must preserve or export them rather than erase them.

8. Audit Strategy

8.1 Which entities create audit events

Every mutating action on the following produces an audit event (BR-14): Organization (create, status changes, settings updates, ownership transfer/recovery), User (invite, accept, suspend, reactivate, deactivate, role change), Invitation (create, revoke, expire), Contract (upload, extract, metadata change, status change, soft delete), ContractNote (add, edit, delete), WitnessInvitation (create, access, revoke, expire), and AIAnalysis (request, complete, fail). Authorized Platform actions that affect a tenant (org create/suspend/archive, ownership recovery, support-access elevation) are logged against that Organization with a PlatformUser actor.

8.2 Fields captured per event

Field	What it captures
id	Unique event identifier.
organization_id	The tenant the event belongs to (scope; BR-18).
actor_type	USER, PLATFORM_USER, or SYSTEM.
actor_user_id / actor_platform_user_id	The responsible actor (one is set, or neither for SYSTEM).
action	The AuditAction value (Section 3.9).
target_entity_type / target_entity_id	What was acted upon (e.g., Contract + its id).
contract_id	Convenience scope for the common per-contract audit view (nullable).
witness_invitation_id	Set on witness-access events, tying the access to a specific link.
old_value / new_value	Redacted JSON before/after snapshots of the changed fields.
ip_address / user_agent	Request context.
created_at	When it happened (UTC). No updated_at or deleted_at — the record is immutable.

8.3 Redaction & immutability

- **Never captured:** password hashes, raw witness tokens, and full contract text are never written into old_value/new_value snapshots. Snapshots capture which fields changed, with sensitive values redacted — echoing the AI module's “no PII leakage” stance.
- **Immutable:** a database rule rejects UPDATE and DELETE on audit_logs; only INSERT is permitted (BR-13). The application has no code path that modifies an existing event.
- **Retained:** audit rows are kept when their referenced Contract is soft-deleted, their User is deactivated, or their Organization is archived.

9. Seed Strategy

The seed produces a coherent, demonstrable environment for development and demos. It is idempotent, uses no real secrets (credentials come from environment configuration), and inserts records in dependency order so that all foreign keys resolve. Seed operations flow through the normal service paths so that they also generate realistic audit history.

Seed item	What is created
Platform Super Admin	One PlatformUser (role SUPER_ADMIN, status ACTIVE), created outside any Organization (BR-21). Enables platform operation from first boot.
Default Organization	One Organization (e.g., "Clausio Demo Co.") progressed to ACTIVE, with its OrganizationSettings (timezone, language, default AI provider, retention policy, branding).
Owner	One User in the default Organization (role OWNER, status ACTIVE); the Organization's owner_user_id points to this User, satisfying BR-1.
Demo Users	One ADMIN and one INTERNAL (both ACTIVE), plus one INVITED user with a matching PENDING Invitation to demonstrate the invitation flow and expiration.
Demo Contracts	Several Contracts spanning both statuses: one DRAFT with processing status PENDING_EXTRACTION, one UNDER_REVIEW with processing status AI_COMPLETED, one COMPLETED, one ARCHIVED, and one soft-deleted; one deliberately has null extracted_text and processing status FAILED to exercise BR-15.
Witness Tokens	WitnessInvitations for a demo Contract in each state: one ISSUED (valid), one USED, one EXPIRED, one REVOKED — storing token_hash only; the raw token is printed once in seed output for testing.
AI Analyses	One each of SUMMARY, RISK, and CLAUSE_QUERY (COMPLETED, with disclaimer text per BR-11 and a pinned model_version), plus one FAILED analysis to show that state.
Audit history	Events emitted naturally by the above operations, so per-contract and per-user audit views have realistic data on first run.

Safety: the seed is intended for non-production environments only and is clearly marked as demo data; it never hardcodes production credentials or real personal data.

10. Prisma Mapping

Each business concept and the future Prisma model that will represent it, with the underlying table name. This is a naming map for implementation — not schema code. Two concepts (Platform and Workspace) have no model at all, which is itself the important design statement.

Business concept	Prisma model	Database table
Platform	— (no model; deployment/tenancy scope)	—
Platform Super Admin / Support Engineer	PlatformUser	platform_users
Organization	Organization	organizations
Organization Settings	OrganizationSettings	organization_settings
Workspace	— (no model; conceptual — the Organization owns all data)	—
User (Owner / Admin / Internal)	User	users
Invitation (member)	Invitation	invitations
Witness	— (no persistent identity)	—
Witness Invitation / Token	WitnessInvitation	witness_tokens
Contract	Contract	contracts
Contract Note	ContractNote	contract_notes
AI Analysis	AIAnalysis	ai_analyses
Audit Log	AuditLog	audit_logs
Tag / Contract-Tag (optional)	Tag / ContractTag	tags / contract_tags

Enums to declare: OrganizationStatus, UserStatus, UserRole, PlatformUserRole, PlatformUserStatus, InvitationStatus, ContractBusinessStatus, ContractProcessingStatus, WitnessStatus, AIAnalysisType, AIAnalysisStatus, AuditAction — plus the optional ContractLegalState and RiskSeverity.

Reminder: *there is deliberately no Workspace model and no Witness identity model. A future developer should not create either; the Organization is the owner of all data, and witness access lives only in WitnessInvitation.*

Appendix A. Reconciliation with the Blueprint

Both source documents are authoritative. Where the later-approved Domain & Business Rules refined the original Blueprint tables, this DDS follows the business rules and records the decision here for sign-off. None of these change the approved architecture; they align the data model to the approved domain.

Topic	Decision
Witness as a user role	The Blueprint's user role enum listed witness. Removed here: a Witness is never a User (BR-4). Witness access is modeled solely by WitnessInvitation. UserRole is OWNER / ADMIN / INTERNAL.
Owner role	The Blueprint had admin/internal only. OWNER is added as a distinct role to support the single-Owner rules (BR-1, BR-2, BR-19).
Organization settings	The Blueprint stored settings as an opaque JSON blob. Refined into a typed OrganizationSettings entity for the named business settings, with room for an open-ended JSON field for extras.
Contract status	Split into two independent enums: ContractBusinessStatus (DRAFT → UNDER_REVIEW → COMPLETED → ARCHIVED) for the workflow a human tracks, and ContractProcessingStatus (PENDING_EXTRACTION → EXTRACTION_COMPLETED → AI_PENDING → AI_COMPLETED, with FAILED) for internal technical operations, so the two can evolve independently. The Blueprint's legal-validity values (Draft/Active/Expired/Terminated) are preserved separately as the optional ContractLegalState, since they describe a third, different axis (real-world legal validity).
New entities	PlatformUser and Invitation are added — required by the approved Platform Roles and invitation-flow sections, though absent from the Blueprint's original seven tables.
Table naming	The business term “Witness Invitation” maps to the Blueprint's existing witness_tokens table to avoid renaming an approved table.
Email uniqueness	Enforced globally per BR-17, rather than per-organization.
Concurrency	Added as an implementation guideline (Section 1.8): optimistic locking via a version field or equivalent on mutable entities, especially Contract. No schema redesign; satisfied by one additional field where needed.
Tenant isolation enforcement	Section 1.2 strengthened to state explicitly that isolation must never rely on frontend filtering; organization scoping must be enforced server-side, in every backend service, repository, and query, for every authenticated request.
Invitation history	Section 2.5 expanded to state explicitly that Invitations are single-use and are always retained as historical records once accepted, expired, or revoked; re-inviting a former member always issues a brand-new Invitation and token, and no old token can ever be reused.

End of document — Clausio Database Design Specification, v1.0